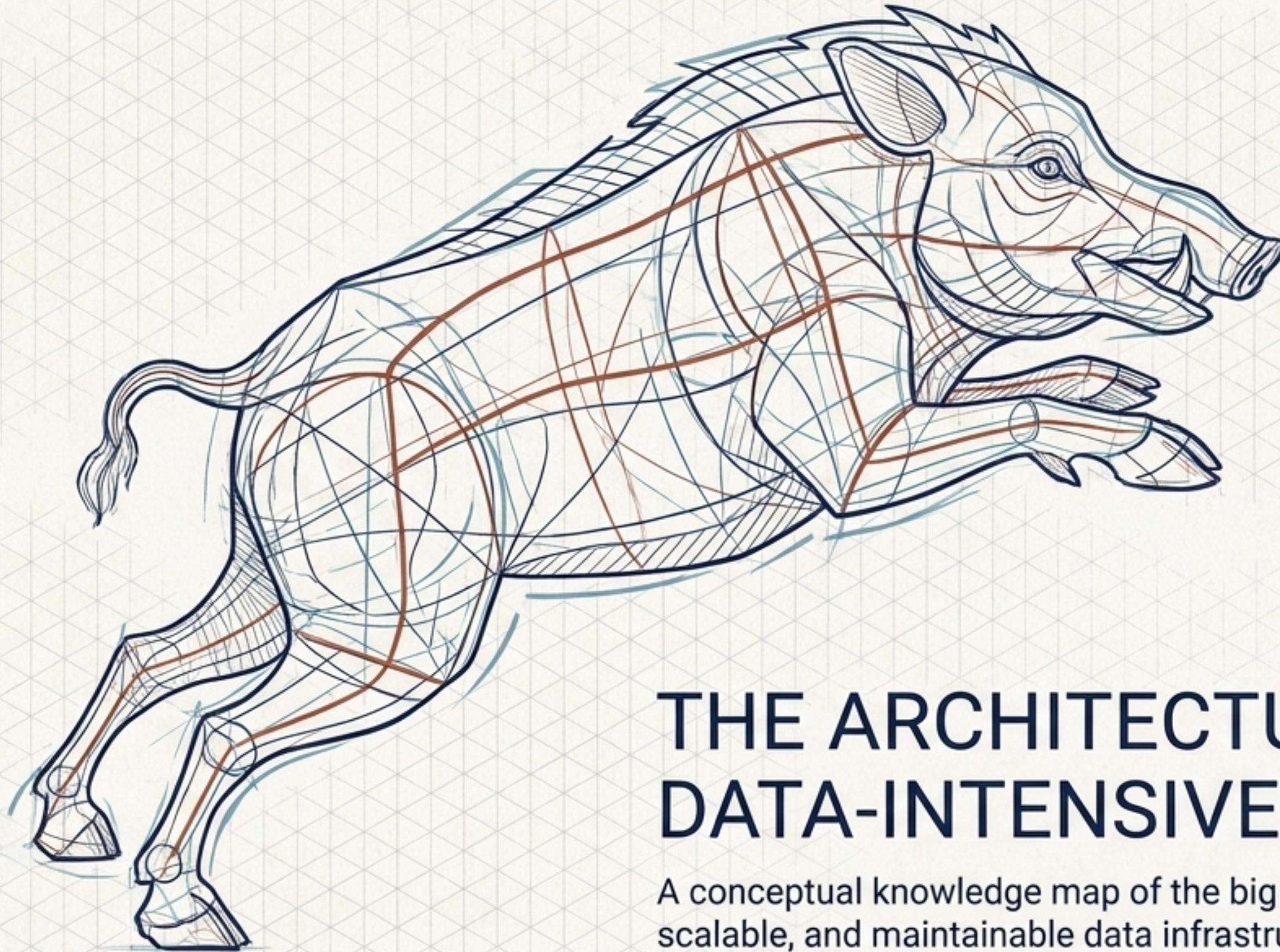
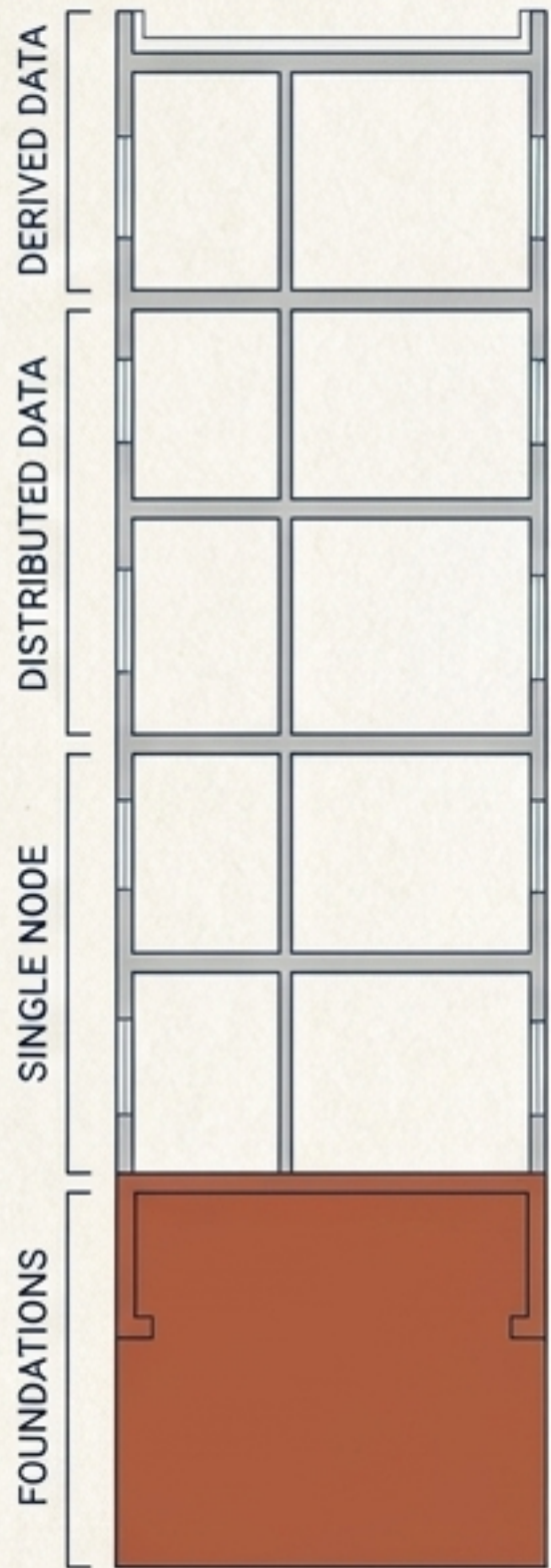




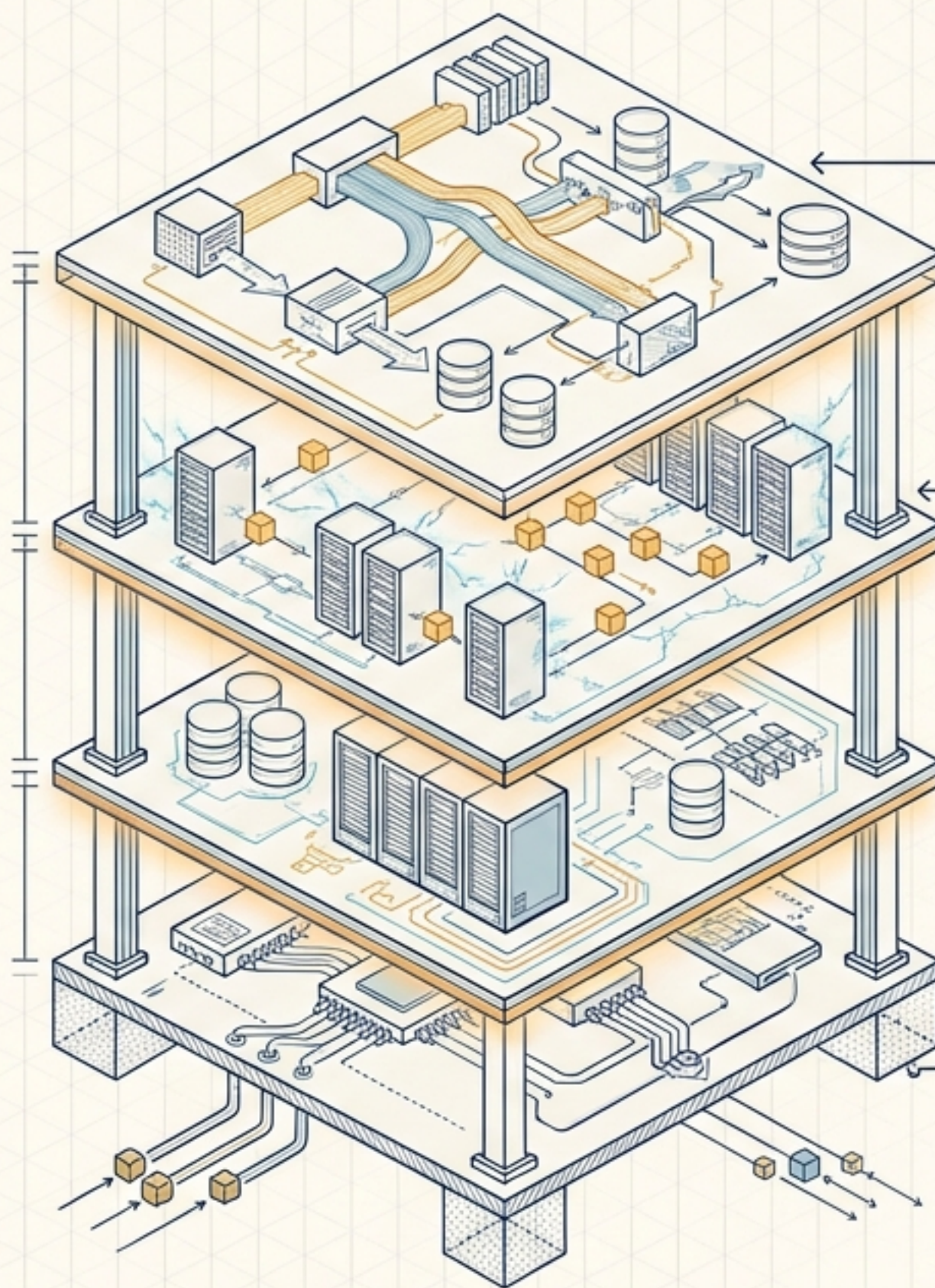
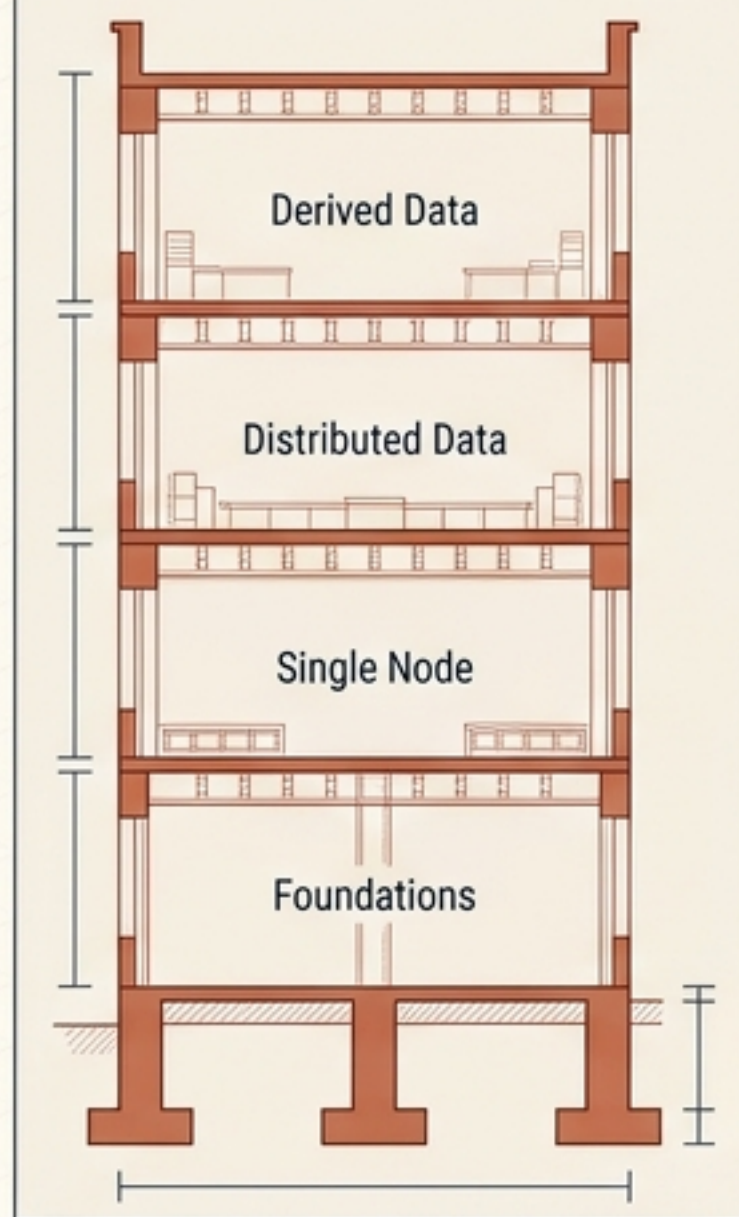
THE ARCHITECTURE OF DATA-INTENSIVE SYSTEMS

A conceptual knowledge map of the big ideas behind reliable, scalable, and maintainable data infrastructure.

Based on the foundational text by Martin Kleppmann and Chris Riccomini.

Four Layers of Data Architecture

ARCHITECTURAL LEGEND



Derived Data

Batch, streams, data pipelines, and unbundling the database.

Distributed Data

Replication, sharding, isolation, and the chaos of the network.

Single Node

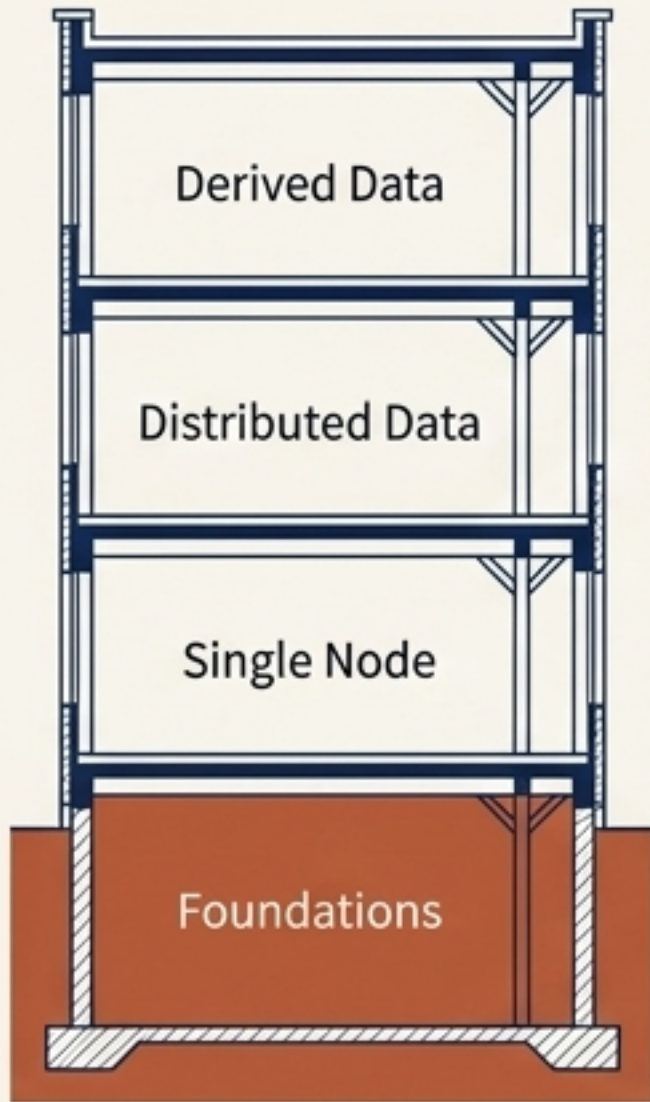
Data models, storage engines, and internal data structures.

Foundations

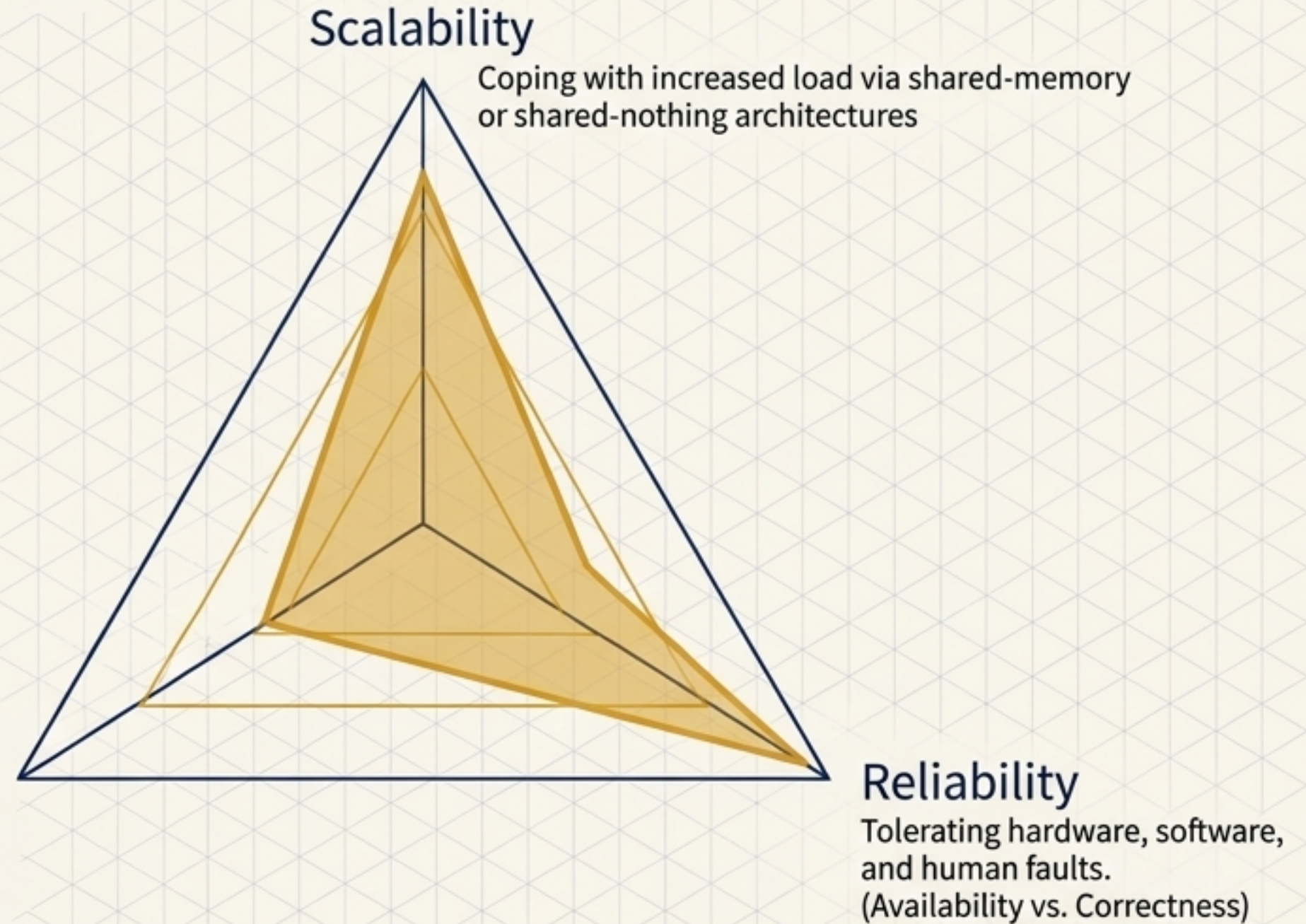
Defining system health and workload trade-offs.

The System Health Rubric

Architectural Legend

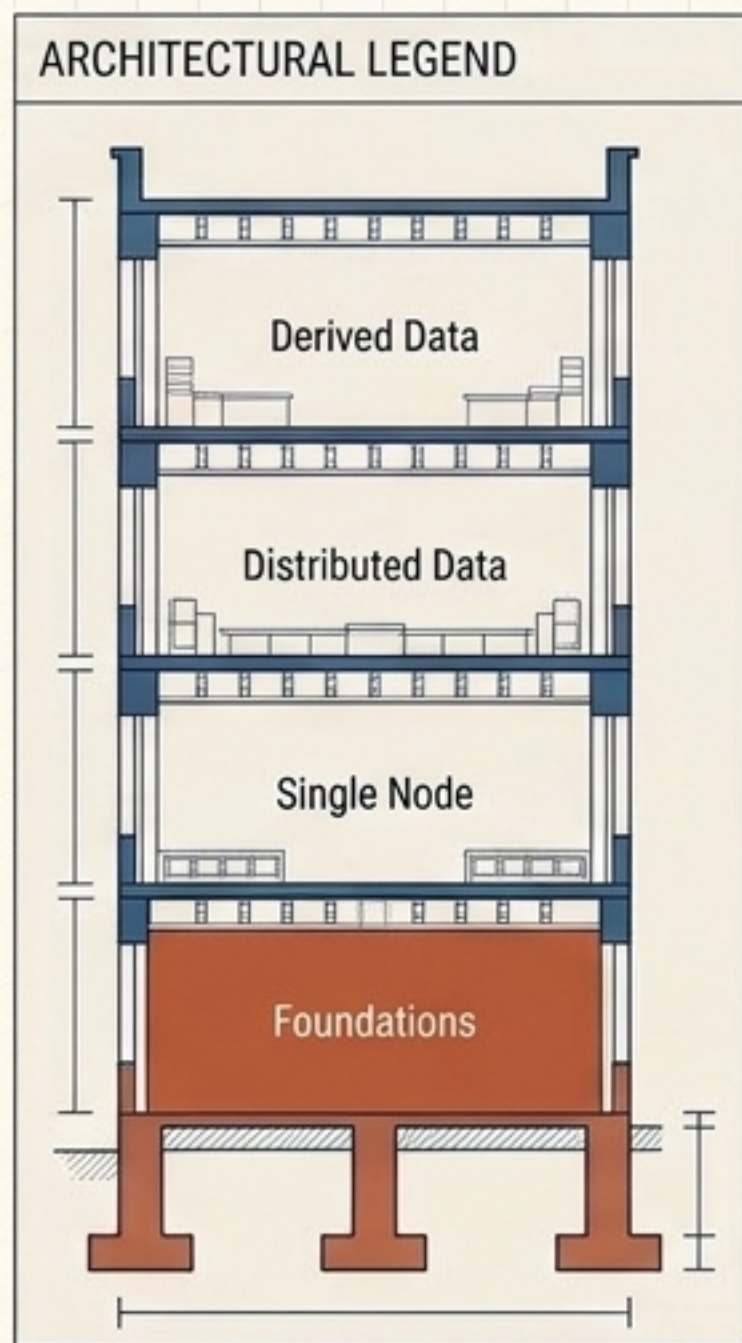


Maintainability
Operability, simplicity, and
evolvability (preventing
the “big ball of mud”)



There are no perfect solutions, only calculated trade-offs.
Optimizing for scale often introduces complexity, degrading maintainability.

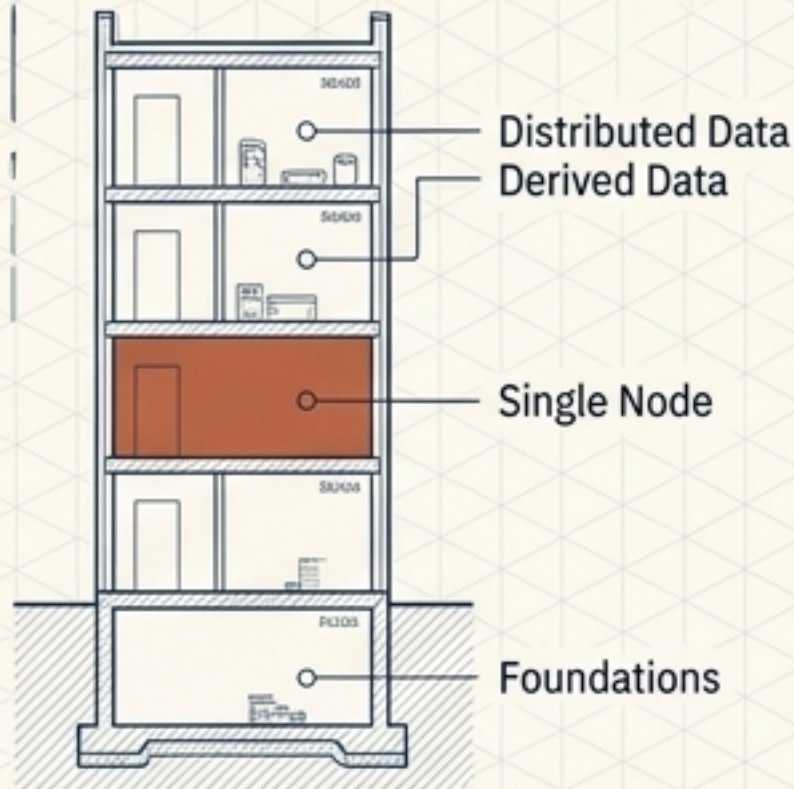
Divergent Workloads: OLTP vs. OLAP



	OLTP	OLAP
Main Read Pattern	Point queries by key	Aggregate over millions of records
Main Write Pattern	Low-latency updates	Bulk imports / ETL
Data Representation	Latest point-in-time state	History of events over time
Typical Bottleneck	Disk seek time	Disk bandwidth / CPU

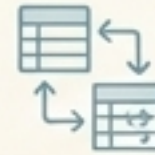
The Data Model Fitness Landscape

ARCHITECTURAL LEGEND



Complexity of Relationships (Low to High)

Relational Model (SQL)
Medium Flexibility,
Medium Relationship
Complexity.
Ideal for normalized
data, joins, and strict
schema-on-write.



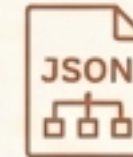
Zone 2

Zone 3



Graph Model (Cypher/SPARQL)

High Flexibility, High
Relationship Complexity.
Ideal for highly interconnected
data where everything
relates to everything.



Document Model (JSON)

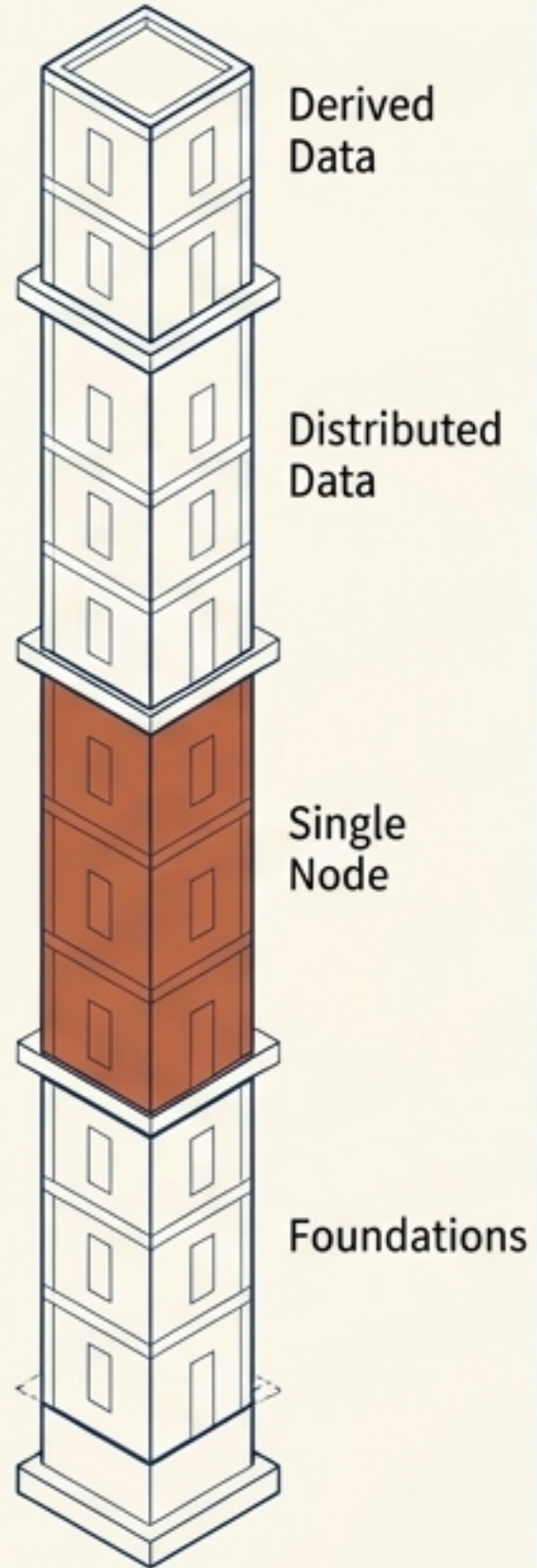
High Flexibility, Low
Relationship Complexity.
Ideal for self-contained,
schema-on-read data.

Zone 1

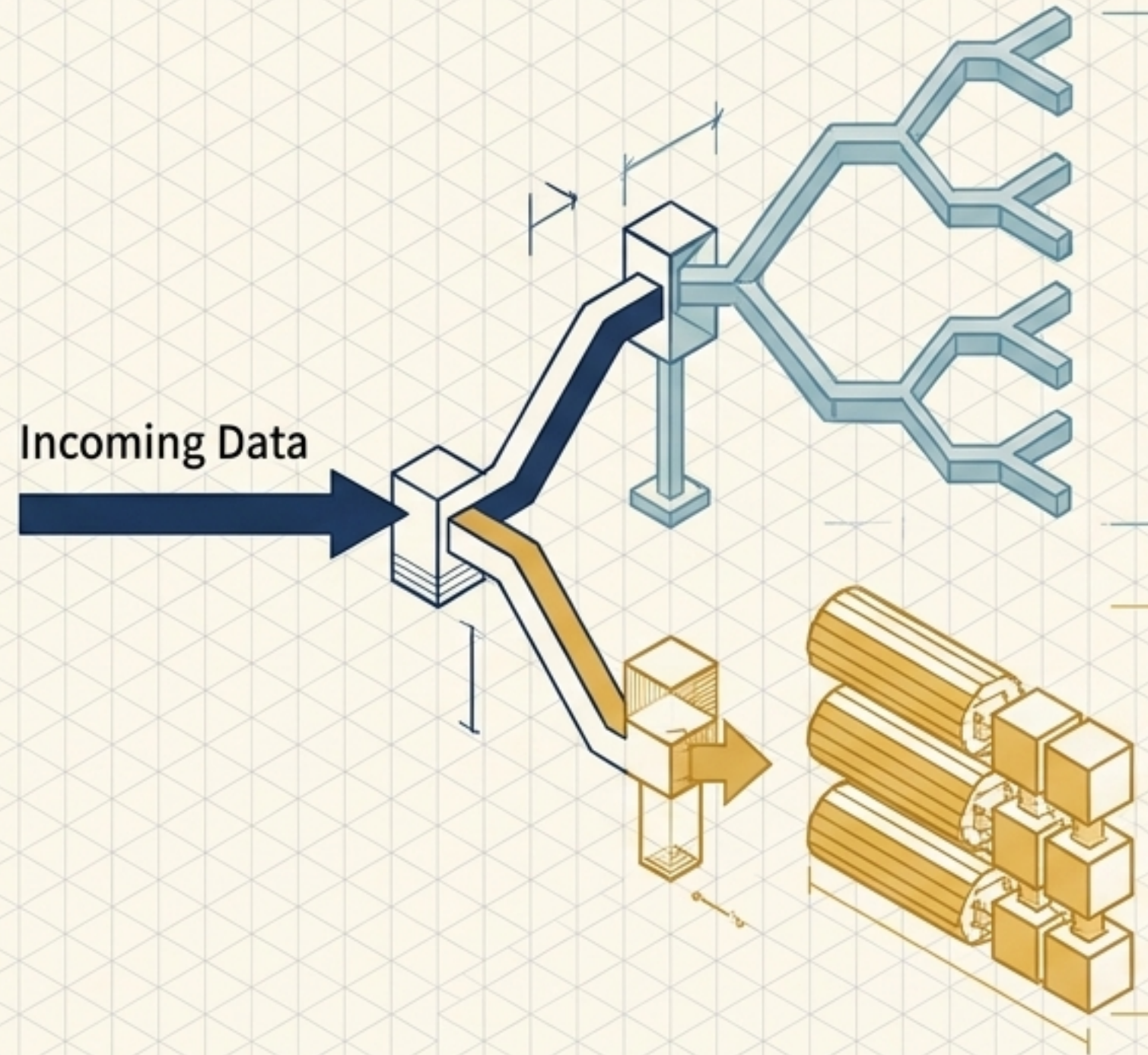
Need for Schema Flexibility (Low to High)

There are no perfect solutions, only calculated trade-offs. Optimizing for scale often introduces complexity, degrading maintainability.

Architectural Legend



The Storage Fork: In-Place vs. Append-Only

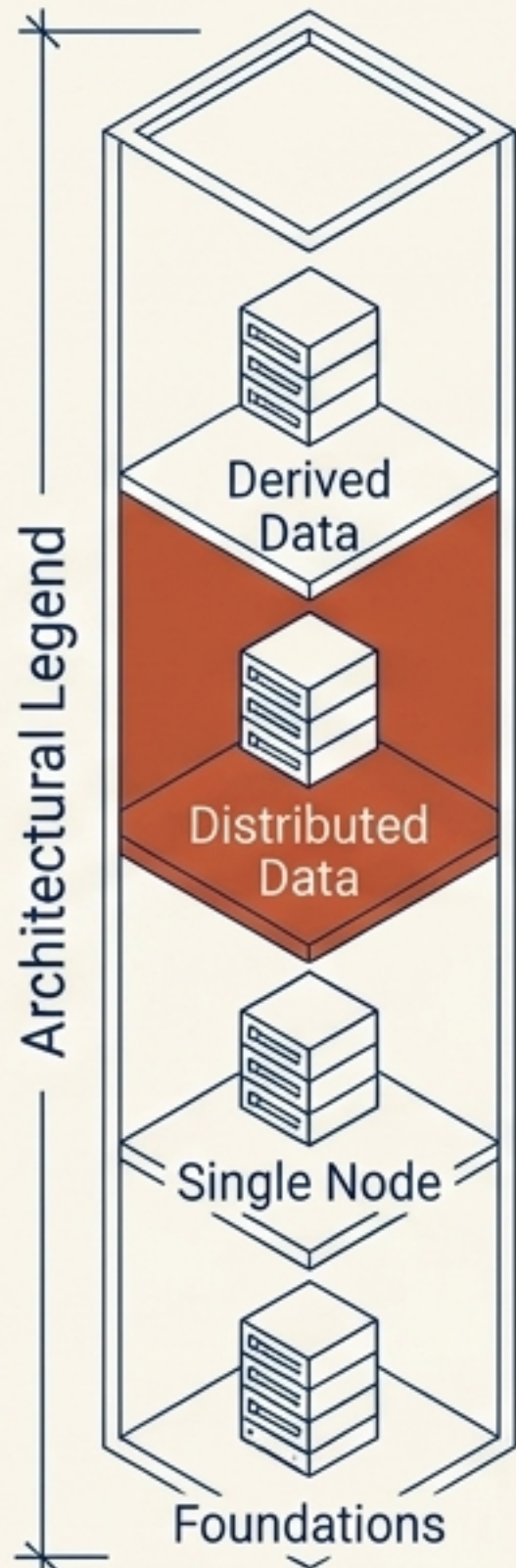


The B-Tree

- Updates data in-place.
- Optimized for read-heavy workloads.
- Tension: High write-amplification (modifying whole blocks for small writes).

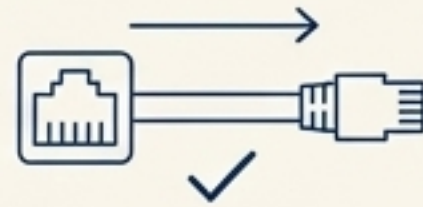
Log-Structured Merge (LSM) Trees

- Appends data continuously to immutable files.
- Optimized for high-throughput writes.
- Tension: High read-amplification (checking multiple files to find a key); relies on background compaction.

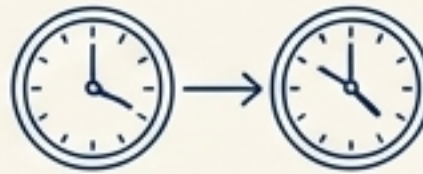


The Distributed Crucible: Assumption vs. Reality

Assumption



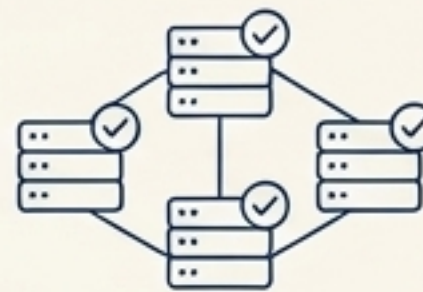
Assumption: The network is reliable.



Assumption: Clocks are synchronized.

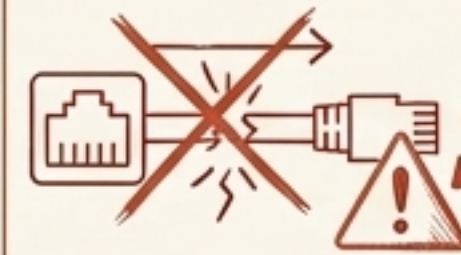


Assumption: Processing is continuous.



Assumption: Nodes are reliable.

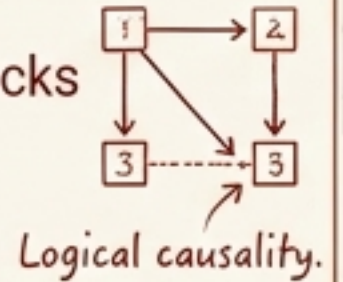
Reality



Reality: Packets drop; unbounded delays cause split brain.



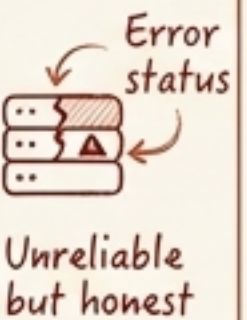
Reality: Time-of-day clocks drift; logical clocks are required for causality.



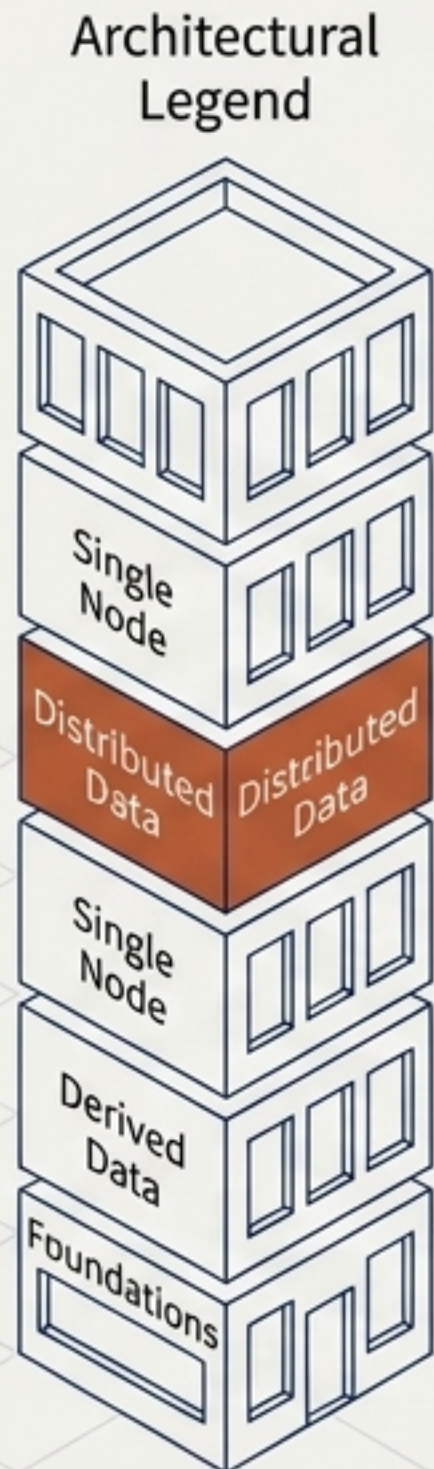
Reality: Garbage collection pauses freeze nodes; leases expire silently.



Reality: Partial failures are inevitable; assume nodes are unreliable but honest (non-Byzantine).

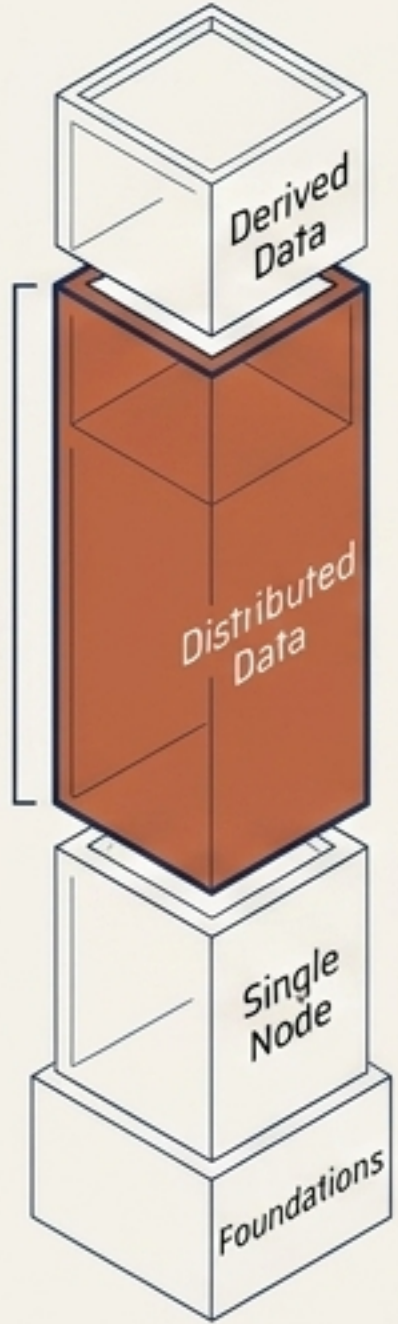


Replication Topologies

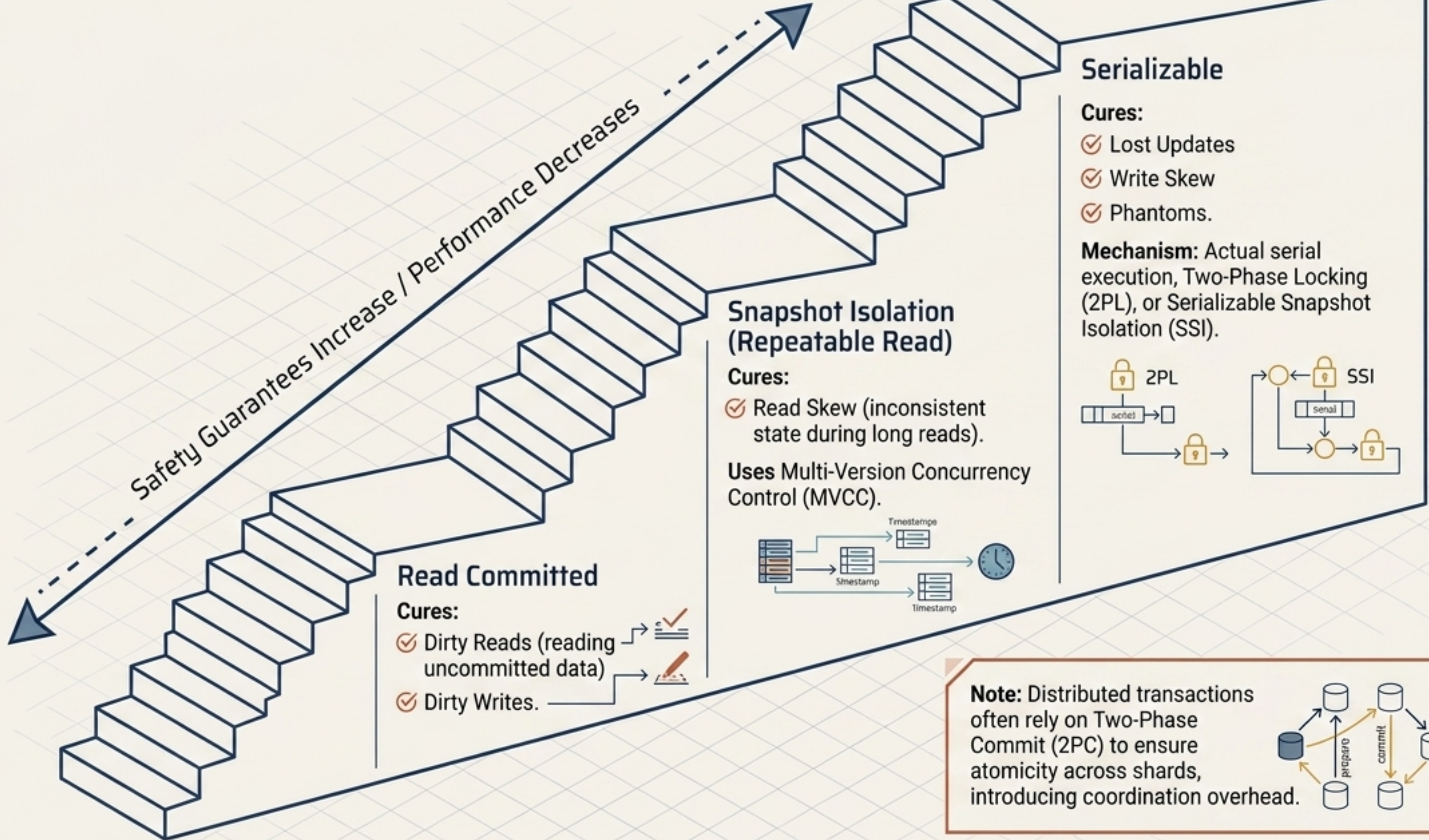


Single-Leader	Multi-Leader	Leaderless (Dynamo-style)
Mechanism: All writes go to one node; followers copy logs. Trade-off: Easy concurrency control, but poor write availability if the leader fails.	Mechanism: Multiple nodes accept writes; sync asynchronously. Trade-off: Excellent write availability (multi-region), but introduces complex conflict resolution.	Mechanism: Clients write to/read from a quorum of nodes. Trade-off: High availability and latency tolerance, but requires read repair and handles concurrent writes poorly.

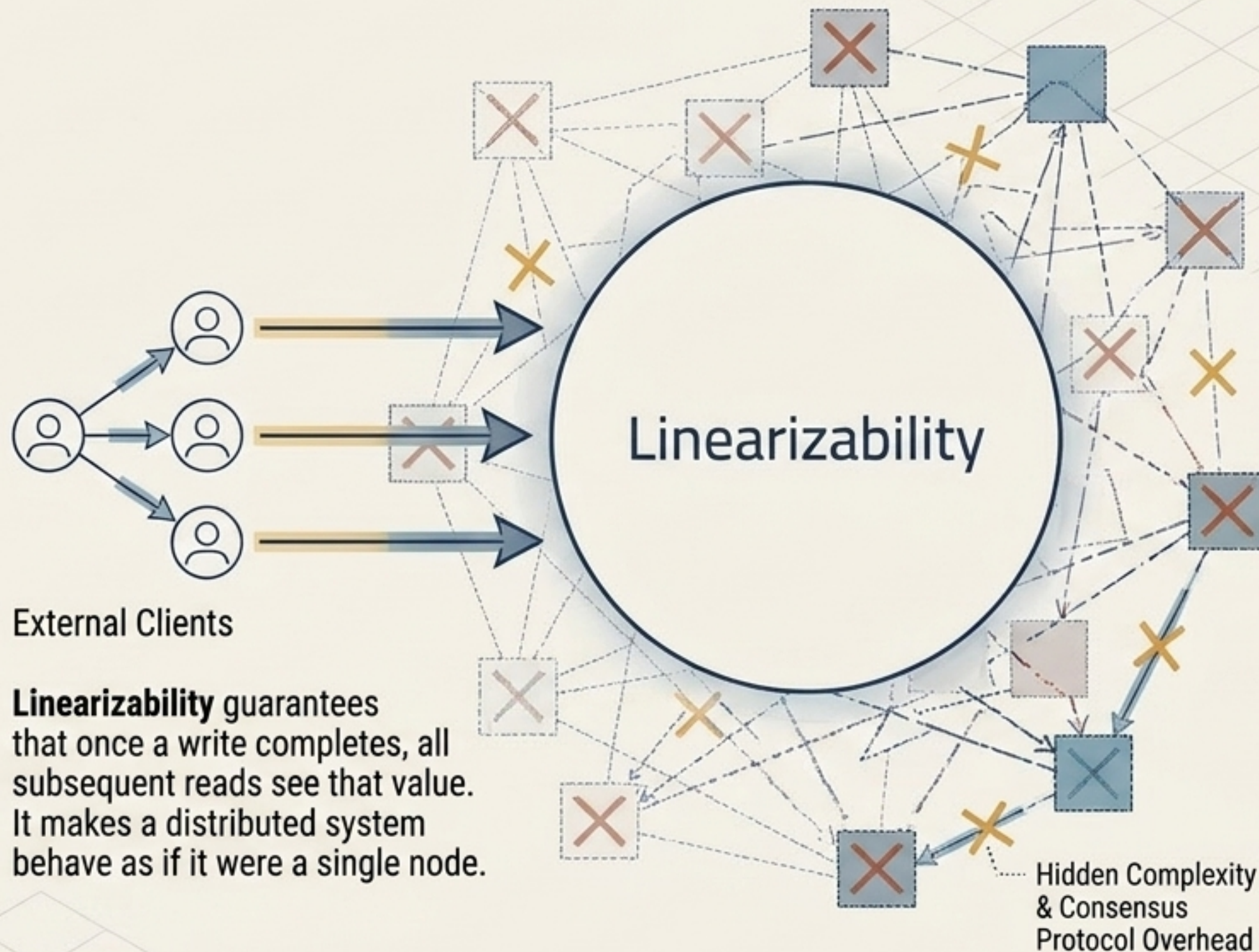
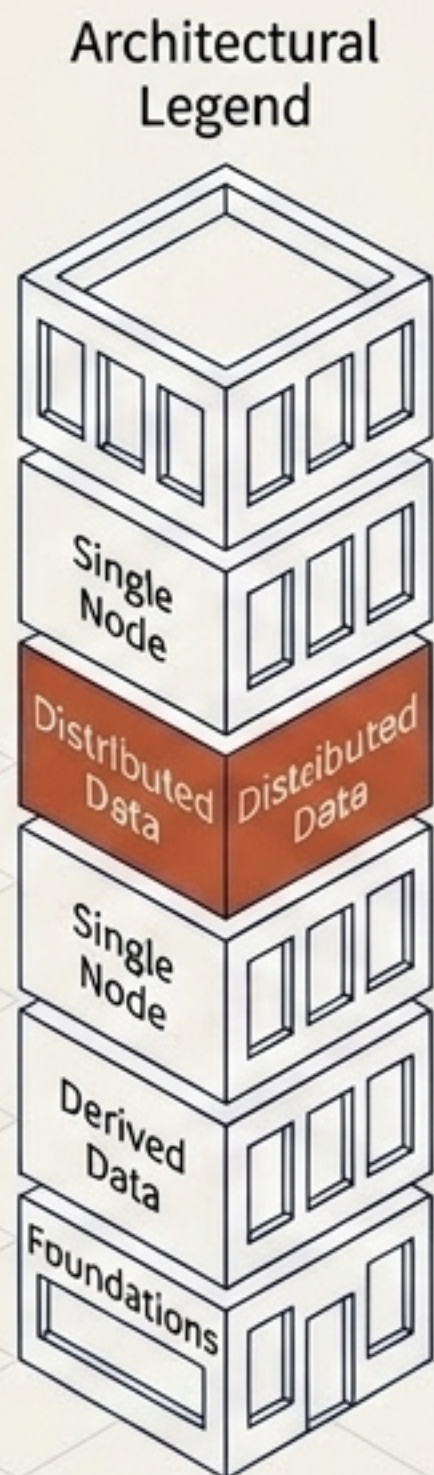
Architectural Legend



The Isolation Staircase



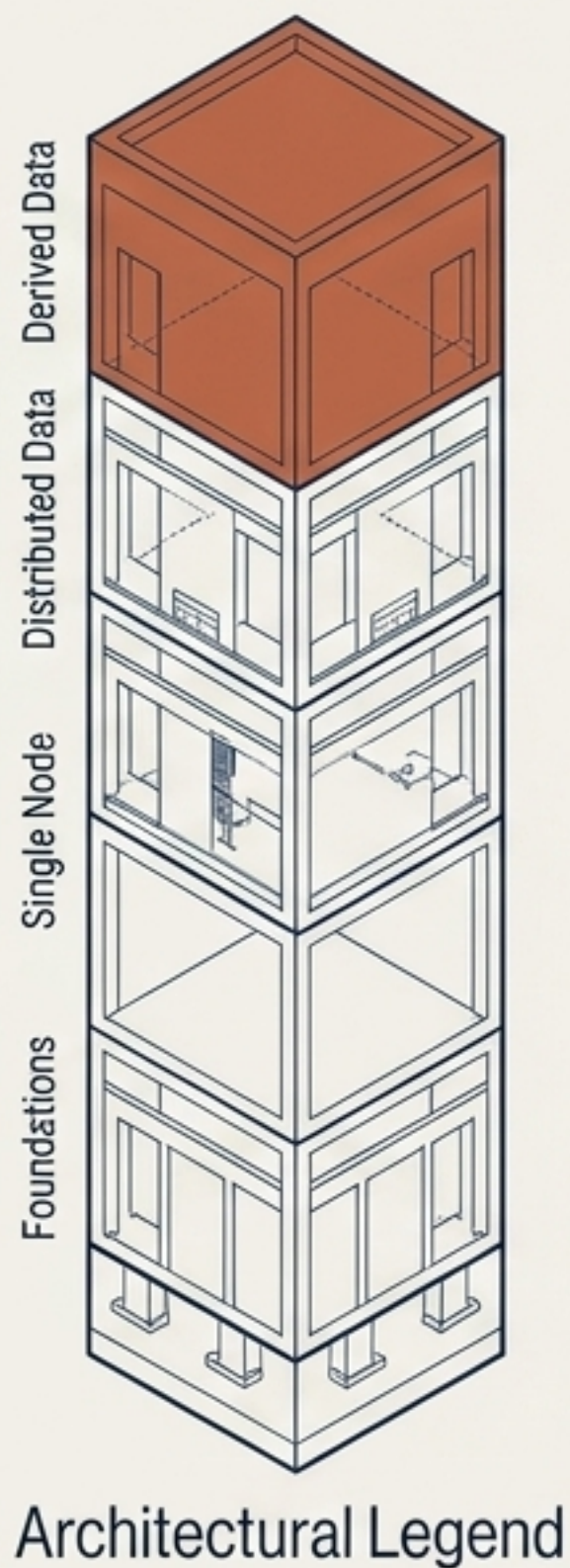
The Consensus Illusion



The Equivalence Principle

All these mechanisms are mathematically equivalent to Consensus (requiring a quorum and a protocol like Raft or Paxos):

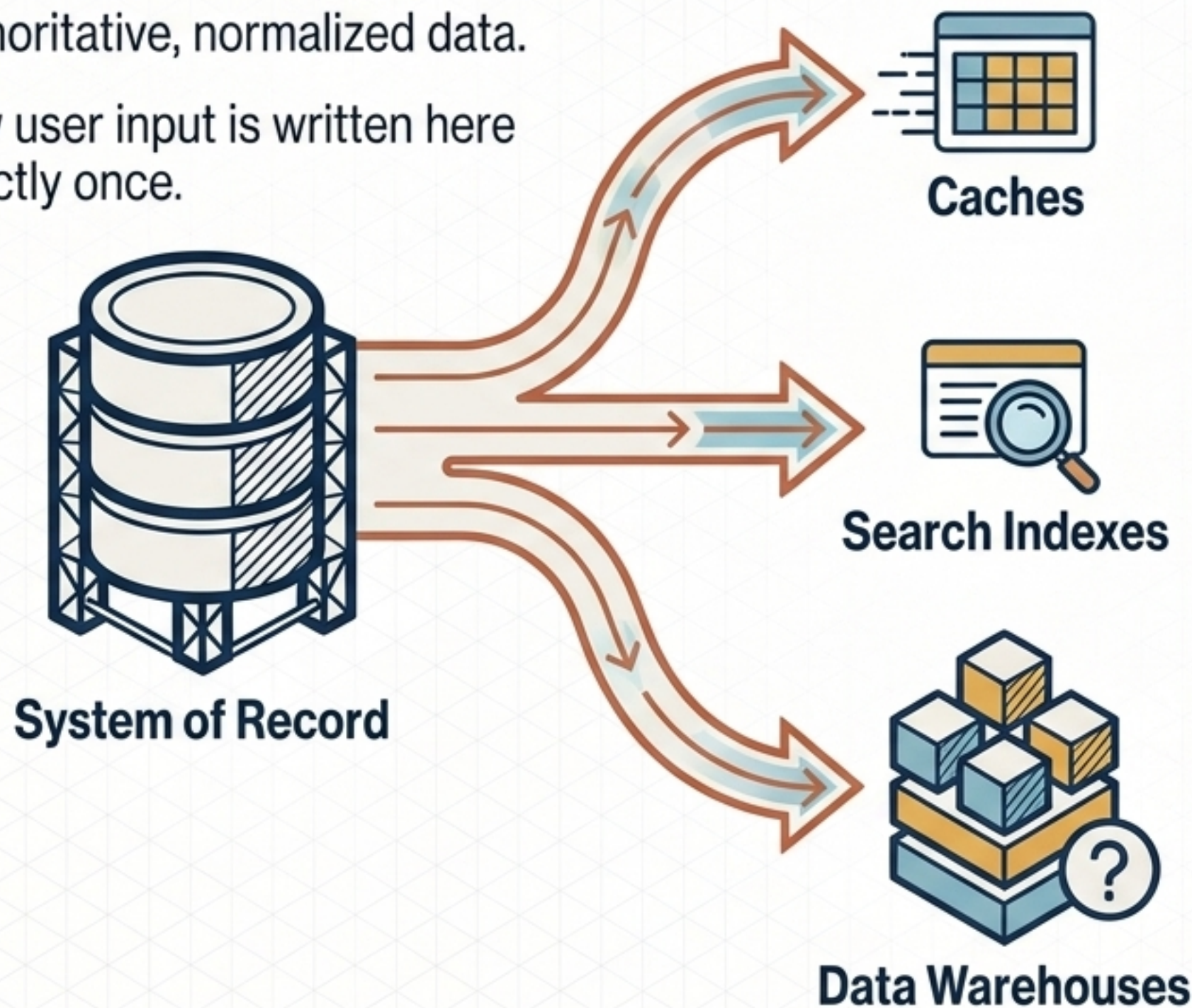
- Linearizable Compare-And-Set (CAS)
- Atomic Transaction Commit (2PC)
- Total Order Broadcast (Shared Logs)
- Uniqueness Constraints



Systems of Record vs. Derived Data

System of Record (Source of Truth):

- Authoritative, normalized data.
- New user input is written here exactly once.



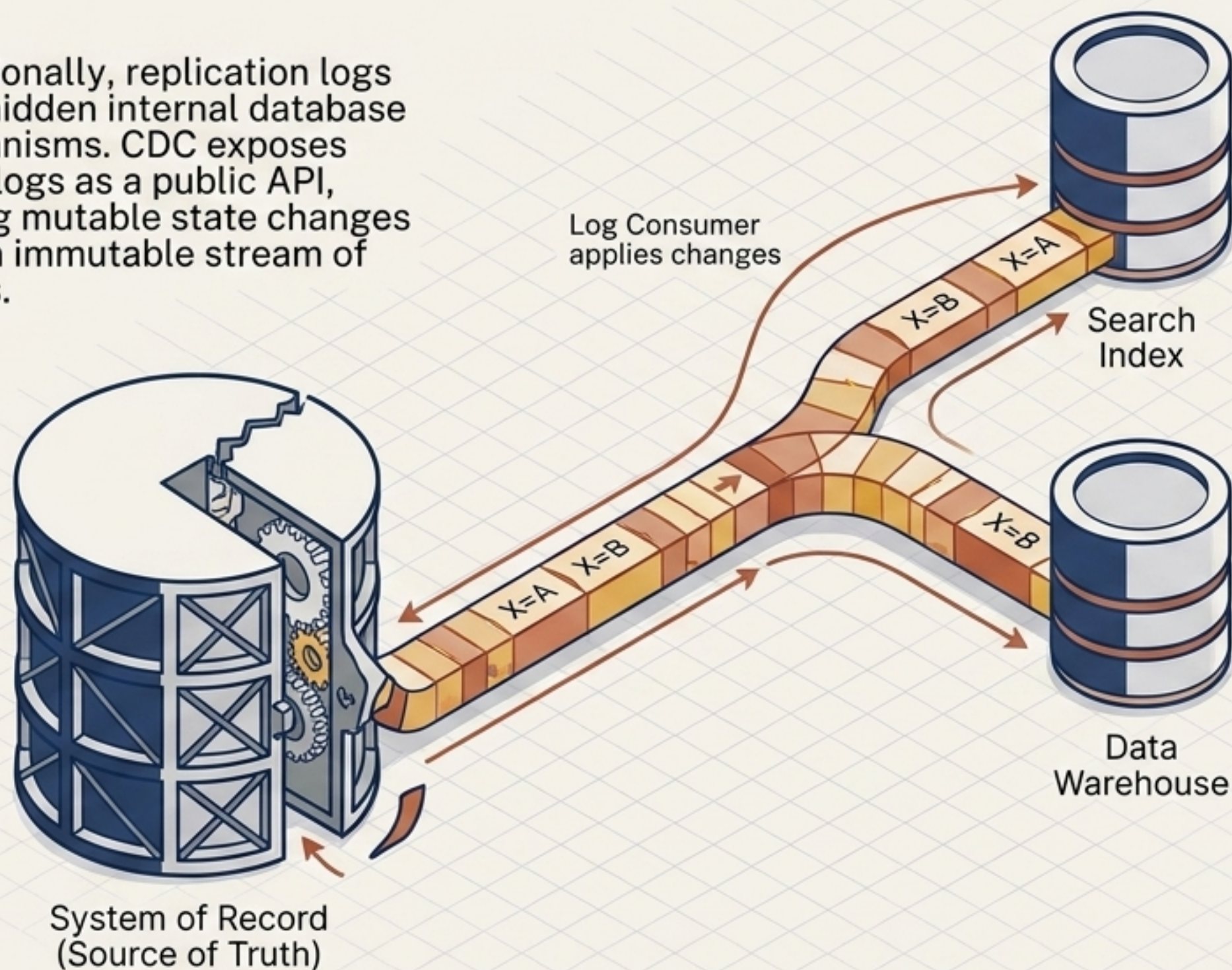
Derived Data Systems:

- Redundant but essential for read performance.
- If lost, can be completely recreated from the source of truth.
- Examples: Materialized views, full-text search indexes, OLAP data warehouses, ML feature vectors.



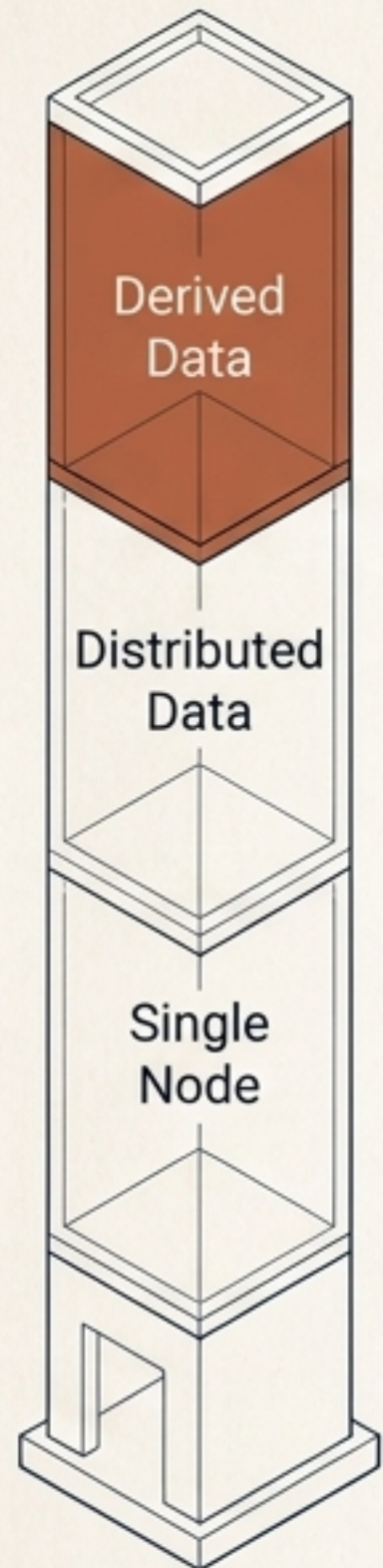
Unlocking the Log: Change Data Capture (CDC)

Traditionally, replication logs were hidden internal database mechanisms. CDC exposes these logs as a public API, turning mutable state changes into an immutable stream of events.



The Paradigm Shift:

Instead of brittle, dual-write operations in application code, downstream systems (indexes, caches) simply subscribe to the CDC event log, ensuring absolute ordering and eventual consistency with the source of truth.



Foundations

The Dual Nature of Data: State and Streams

$$state(now) = \int_{t=0}^{now} stream(t) dt$$

$$stream(t) = \frac{d\ state(t)}{dt}$$

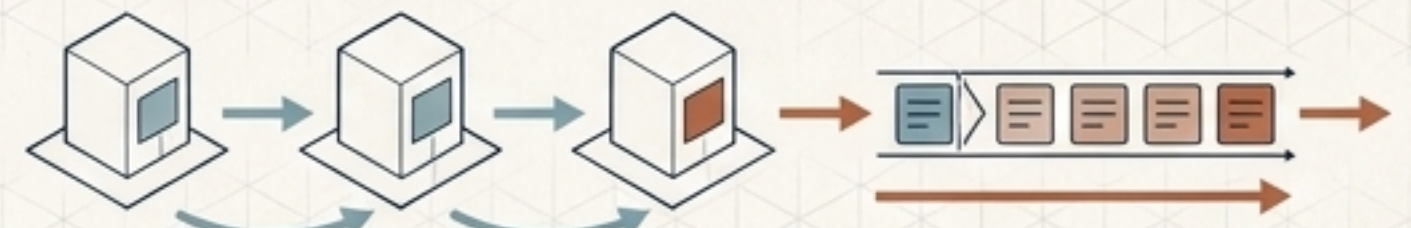
State is the Integral of the Stream:

State represents the current worldview. It is computed by integrating (summing up) all events from the beginning of time to *now*. (Batch Processing / Event Sourcing).



The Stream is the Derivative of State:

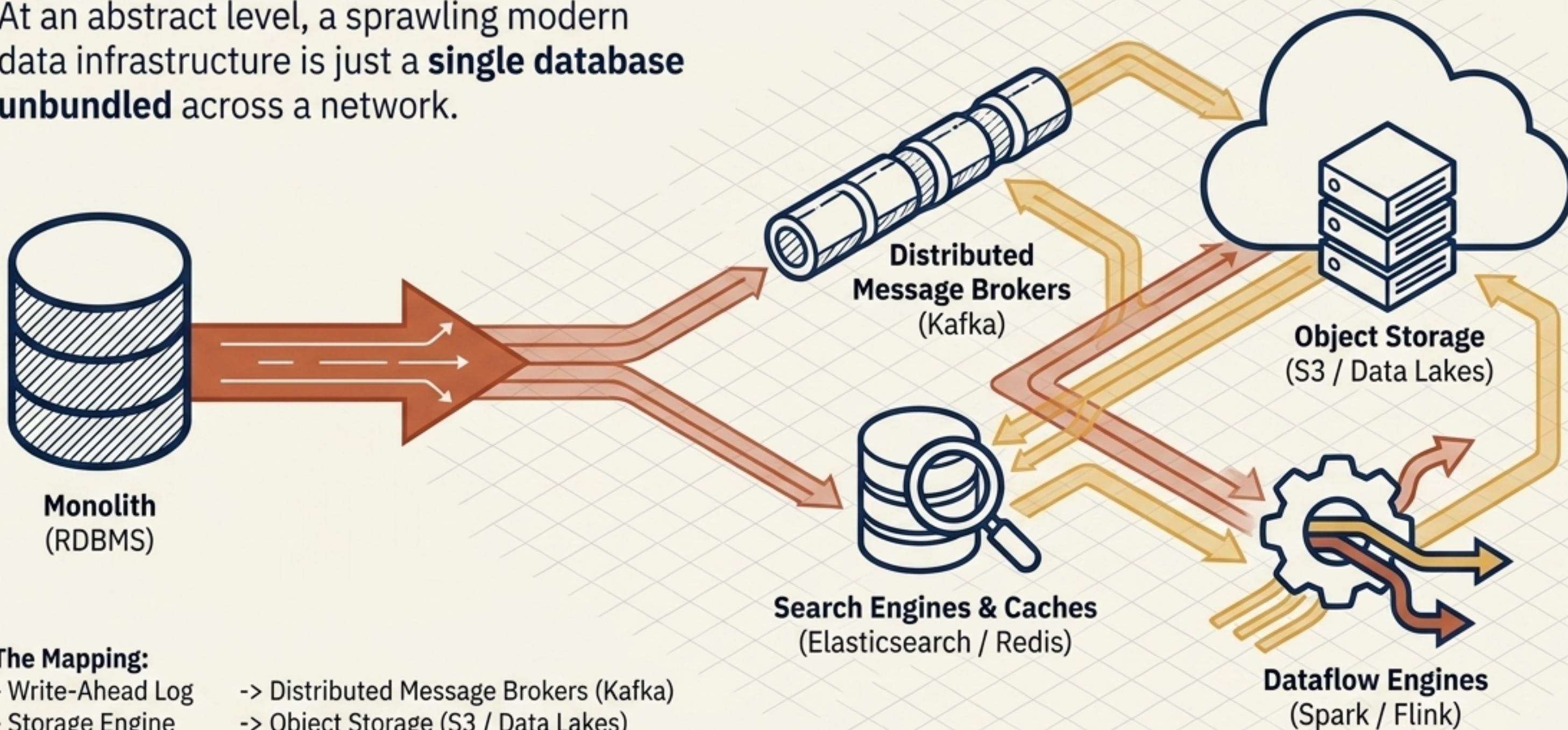
A stream represents the rate of change. It captures how the state transitions from one moment to the next. (Stream Processing / CDC).



Conclusion: Mutable state and append-only logs are two sides of the same coin.

Synthesis: Unbundling the Database

At an abstract level, a sprawling modern data infrastructure is just a **single database unbundled** across a network.

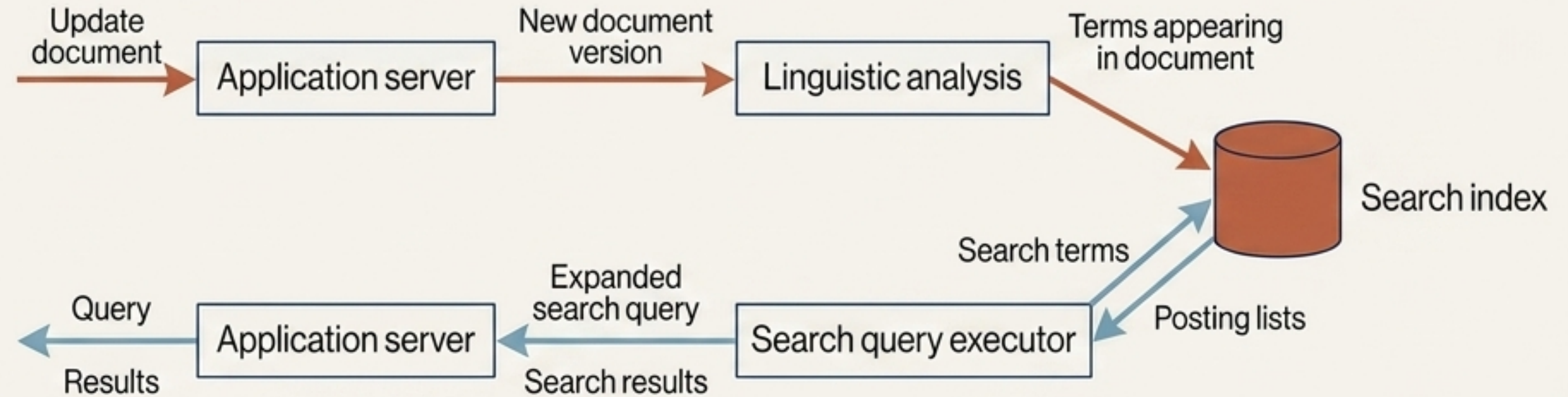


The Mapping:

- | | |
|---------------------|--|
| - Write-Ahead Log | -> Distributed Message Brokers (Kafka) |
| - Storage Engine | -> Object Storage (S3 / Data Lakes) |
| - Secondary Indexes | -> Search Engines & Caches (Elasticsearch / Redis) |
| - Query Optimizer | -> Dataflow Engines (Spark / Flink) |

Result: Dataflow becomes the meta-database.

The Intersection of Intent: Write Path meets Read Path



The Write Path (Eager)

Precomputes and derives data as events arrive, regardless of whether anyone asks to see it.
Pushes state forward.

The Read Path (Lazy)

Triggers only when a user requests data.
Pulls state backward.

The Ultimate Trade-off:

System architecture is the art of shifting the boundary between these two paths. Do you do the heavy lifting at write time (stream processing) or read time (batch analytics)? Complex systems that work evolve from simple systems that work. Embrace the log, acknowledge the trade-offs, and master the dataflow.

